OFFICIAL ENGINEERING & ARCHITECTURE MANUAL

SAKOON TEMPLE WEB PLATFORM

Detailed Logic Blueprint: Autonomous Lunisolar Astronomical Engine, Real-Time Clock Scheduler, UI State Systems & CI/CD

Architecture: Angular 21 (Standalone + Signals) **Design:** Pure Vanilla CSS + Responsive Glassmorphism

Build Engine: Vite / esbuild (@angular/build:application) **Hosting:** Netlify CI/CD (Node 22) **Repository:** github.com/ajay78144/temple.git

1. Executive Summary & Core Technologies

The **Sakoon Temple (सकून मंदिर)** web platform is a high-performance, spiritually authentic digital portal designed for devotees worldwide. It provides sanctified services including daily Darshan, live Aarti schedules, Vedic astrology (Kundali), Tarot energy readings, Maha Yagya / Havan booking, Tarpan / Pitru Shradh rites, and personalized family milestone ceremonies.

Domain	Technology	Engineering Role & Justification
Core Framework	Angular 21 (Standalone)	Lightweight, tree-shakable standalone architecture eliminates legacy NgModules, reducing initial bundle weight to ~125 KB transfer size.
Reactivity Engine	Angular Signals (`signal` , `computed`)	Fine-grained reactive state without Zone.js overhead. State changes in calendar year, month, or filters recompute instantly.
Panchang Algorithm	Autonomous Astronomical Math (In-house)	Eliminates 3rd-party API dependencies. Computes planetary degrees, Tithis, Nakshatras, and 100+ festivals client-side across 2020–2045+.
Design & Styling	Pure Vanilla CSS 3	Zero external utility dependencies. Employs CSS custom properties, backdrop-filter glassmorphism, golden shimmer animations, and fluid CSS grids.
Single Page Routing	Angular Router	Zero-refresh transitions with programmatic scroll restoration and active route matching.
Deployment / CI/CD	Netlify + GitHub Webhooks	Automated git push triggers. Configured with `netlify.toml` , Node 22 runtime, and `/* /index.html 200` rewrite rule.

2. High-Level Architecture & Information Flow

The system is designed with a unidirectional, service-driven data flow. The singleton `TempleDataService` serves as the single source of truth for both the astronomical calendar math and the real-time temple schedule state.

```
[Client Browser] | ├──► HeaderComponent (Sticky Scroll Listener & Golden Shimmer Beam) | ├──► HomeComponent (2-Second Circular Slider + Live Status Banner) | ├──► CalendarComponent ──(Queries Signal)──► TempleDataService | ▲ | | └──────(Reactive computed daysInView)──────┘ | | | ┌──────────────────────────────────────┴──────────────────────────────────────┐ | ▼ ▼ | [Autonomous Lunisolar Engine] [1-Second Interval Clock] | • Julian Day Epoch J2000 • totalMins = (hh * 60) + mm | • Sun & Moon Mean Longitudes • Evaluates 9 Temple Slots | • 12° Elongation = Tithi • Calculates 'Next Event' | • 13°20' Division = Nakshatra • Updates liveStatus Signal | • Lunisolar Festival Mapper | | ▲ | | └──────────────────────(Emits To UI)──────────────────────────────┘ | └──► Booking & Contact Forms (Two-Way Reactive Validation & Confirmation Modal)
```

3. The Autonomous Lunisolar Panchang Engine — Deep Logic & Code

Standard web calendars rely on external REST APIs for Hindu Panchang data. These APIs suffer from rate-limiting, latency, downtime, or billing costs. To ensure **100% autonomy, offline reliability, and instant response times**, Sakoon Temple features an embedded mathematical astronomical engine inside `TempleDataService`.

3.1 Mathematical Foundation (Astronomical Equations)

The engine converts any Gregorian date (year, month, day) into solar and lunar coordinates using standard celestial mechanics:

1. Julian Date & J2000 Epoch Conversion

Converts UTC timestamp into continuous astronomical Julian Days (JD), measuring days elapsed since standard Epoch J2000.0:

```
// 06:00 UTC corresponds to approx Indian Sunrise
const timeMs = Date.UTC(year, month, day, 6, 0, 0);
const jd = (timeMs / 86400000.0) + 2440587.5;
const dFromJ2000 = jd - 2451545.0;
```

2. Solar & Lunar Mean Longitude

Computes the geocentric ecliptic longitudes (0° to 360°) of the Sun and Moon:

```
// Mean Sun Longitude (rate ~0.9856°/day)
const sunMean = (280.460 + 0.9856474 * dFromJ2000) % 360;

// Mean Moon Longitude (rate ~13.1764°/day)
const moonMean = (218.316 + 13.176396 * dFromJ2000) % 360;

const sunLong = (sunMean + 360) % 360;
const moonLong = (moonMean + 360) % 360;
```

3.2 Tithi & Paksha Determination Logic

In Vedic astronomy (Surya Siddhanta), a **Tithi** is the time required for the angular distance between the Moon and Sun to increase by 12 degrees. The engine evaluates:

```
// 1. Angular separation (Elongation) between Moon and Sun [0° to 360°)
const elongation = (moonLong - sunLong + 360) % 360;

// 2. Tithi Index (1 to 30)
const tithiIndex = Math.floor(elongation / 12) + 1;

// 3. Paksha & Tithi Name Assignment
if (tithiIndex ⩽ 15) {
  paksha = 'Shukla';        // Waxing Moon (Bright Fortnight)
  tithiNumber = tithiIndex;
  tithiName = tithiNames[tithiIndex]; // Pratipada ... Purnima (15)
} else {
  paksha = 'Krishna';       // Waning Moon (Dark Fortnight)
  tithiNumber = tithiIndex - 15;
  tithiName = (tithiNumber ⩵ 15) ? 'Amavasya' : tithiNames[tithiNumber];
}
```

3.3 Nakshatra Division (27 Lunar Mansions)

The 360° celestial zodiac is divided into 27 equal parts of **13° 20' (13.333333°)** each. The Moon's longitude directly determines the active Nakshatra:

```
const nakshatraIndex = Math.floor(moonLong / (360 / 27));
const activeNakshatra = nakshatras[(nakshatraIndex + 27) % 27];
// Yields: Ashwini, Bharani, Krittika, Rohini ... Revati
```

3.4 Vikram Samvat & Hindu Lunar Month (Masa) Calculation

The Vikram Samvat year precedes the Gregorian year by 57 years (or 56 before Chaitra Shukla Pratipada). The Hindu lunar month is calculated from solar entry into tropical Rashis:

```
const dayOfYear = Math.floor((date.getTime() - new Date(year, 0, 1).getTime()) / 86400000);
const masaIndex = Math.floor(((dayOfYear + 285) % 365) / 30.4375);
const hinduMonth = hinduMonthsList[masaIndex % 12]; // Chaitra to Phalguna

// Vikram Samvat rollover at Hindu New Year (Chaitra Navratri ~March 22)
const vikramSamvat = (month > 2 || (month ⩵ 2 && day ⩾ 22)) ? year + 57 : year + 56;
```

3.5 100+ Festivals & Vrats Autonomous Matching Logic

Festivals are automatically derived by combining `(hinduMonth + paksha + tithiNumber)`. This ensures every festival lands on its exact canonical lunar day for any year:

Festival / Vrat	Astronomical Condition	Algorithm Mapping Rule
24 Ekadashi Vrats	`tithiNumber === 11`	Dynamically maps Hindu Month + Paksha to exact canonical name (e.g., Jyeshtha Shukla = Nirjala, Ashadha Shukla = Devshayani, Kartika Shukla = Dev Uthani).
Pradosh Vrats	`tithiNumber === 13`	Prefixed with day-of-week name (e.g., Tuesday = Bhauma Pradosh, Saturday = Shani Pradosh, Monday = Soma Pradosh).
Maha Shivratri	`Month: Phalguna, Paksha: Krishna, Tithi: 14`	Detects Chaturdashi preceding Phalguna Amavasya; initiates midnight Shiva Pujan.
Krishna Janmashtami	`Month: Bhadrapada, Paksha: Krishna, Tithi: 8`	Ashtami Rohini appearance of Bhagwan Krishna.
Diwali & Deepawali	`Month: Kartika, Paksha: Krishna, Tithi: 15 (Amavasya)`	Maha Lakshmi Pujan and universal illumination.
Sharad Purnima	`Month: Ashwina, Paksha: Shukla, Tithi: 15 (Purnima)`	Full Moon of 16 divine kalas; amrit kheer night.
Rahu Kaal & Muhurat	`date.getDay() (0 to 6)`	Indexed lookup array assigning authoritative Rahu Kaal window per weekday.

3.6 Dynamic Calendar Grid Construction & Date Math

To render an aesthetically balanced 35 or 42 card calendar grid, the `generatePanchangMonth(year, monthIndex)` method executes the following multi-step algorithm:

```
// Step 1: Detect days in target month and day of week for Day 1
const daysInMonth = new Date(year, monthIndex + 1, 0).getDate();
const firstDayIndex = new Date(year, monthIndex, 1).getDay(); // 0=Sun, 1=Mon...

// Step 2: Previous Month Padding (Fills leading cells with muted previous month dates)
const prevMonthDays = new Date(year, monthIndex, 0).getDate();
for (let i = firstDayIndex - 1; i >= 0; i--) {
  const d = new Date(year, monthIndex - 1, prevMonthDays - i);
  result.push(this.calculatePanchangForDate(d, false, today));
}

// Step 3: Current Month Active Days (1 to 28/29/30/31)
for (let day = 1; day <= daysInMonth; day++) {
  const d = new Date(year, monthIndex, day);
  result.push(this.calculatePanchangForDate(d, true, today));
}

// Step 4: Next Month Padding (Completes 7-column grid structure)
const remaining = (7 - (result.length % 7)) % 7;
for (let nextDay = 1; nextDay <= remaining; nextDay++) {
  const d = new Date(year, monthIndex + 1, nextDay);
  result.push(this.calculatePanchangForDate(d, false, today));
}
```

4. Real-Time Temple Clock & Status Logic

Temple rituals and deity rest hours (Vishram) operate on a rigorous temporal schedule. To provide devotees with immediate awareness of temple status, `TempleDataService` maintains an automated 1-second pulse.

4.1 1-Second Interval Loop & Minute Normalization

Rather than parsing complex string dates every frame, the engine converts current time into **Total Minutes of the Day (0 to 1439)**:

```
private updateLiveStatus(): void {
  const now = new Date();
  const hours = now.getHours();
  const mins = now.getMinutes();
  const totalMins = hours * 60 + mins; // 0 to 1439 minutes range

  // Evaluates Temple Operating Ranges
  const isMorningOpen = totalMins ⩾ 330 && totalMins < 750;  // 05:30 AM to 12:30 PM
  const isEveningOpen = totalMins ⩾ 990 && totalMins < 1290; // 04:30 PM to 09:30 PM
  const isOpen = isMorningOpen || isEveningOpen;
  ...
}
```

4.2 Schedule Mapping & Lookahead Event Predictor

The algorithm compares `totalMins` against temple milestones to project the next upcoming event and display active status banners:

Time Window	Minute Range	Temple Status	Predicted Next Event
05:30 AM – 06:30 AM	330 – 390	OPEN NOW Morning Darshan	Mangala Aarti at 06:00 AM
06:30 AM – 12:00 PM	390 – 720	OPEN NOW Archana & Puja	Rajbhog Aarti & Bhog at 12:00 PM
12:00 PM – 12:30 PM	720 – 750	OPEN NOW Rajbhog Aarti	Afternoon Vishram Closing at 12:30 PM
12:30 PM – 04:30 PM	750 – 990	CLOSED (VISHRAM) Afternoon Rest	Evening Reopening at 04:30 PM
04:30 PM – 07:00 PM	990 – 1140	OPEN NOW Evening Darshan	Maha Sandhya Aarti at 07:00 PM
07:00 PM – 09:15 PM	1140 – 1275	OPEN NOW Night Darshan	Shayan Aarti at 09:15 PM
09:30 PM – 05:30 AM	1290 – 1439 (or < 330)	CLOSED FOR NIGHT Serene Slumber	Morning Suprabhatam at 05:30 AM

5. Interactive UI & Animation Logics

✨ 5.1 Sticky Header & Scroll Detection

The `HeaderComponent` tracks viewport scroll via native event listeners. When scroll distance exceeds 50px, it applies the `.header-scrolled` class:

```
// In Header Component
@HostListener('window:scroll', [])
onWindowScroll(): void {
  const offset = window.pageYOffset || document.documentElement.scrollTop;
  this.isScrolled.set(offset > 50);
}
```

This transitions the header into a backdrop-blurred glass panel with an animated golden shimmer beam along the lower edge.

🐂 5.2 2-Second Circular Slider Logic

The `HomeComponent` hero section cycles through sanctum slide images every 2000 milliseconds using circular array modulo indexing:

```
private startAutoSlide(): void {
  this.slideTimer = setInterval(() => {
    this.currentSlide.update(
      idx => (idx + 1) % this.slides.length
    );
  }, 2000);
}
```

Devotees can manually pause, resume, or navigate via indicator dots and arrow buttons.

6. Form Validation, State Management & Production Deployment

6.1 Devotee Booking & Consultation Form Architecture

The `BookingComponent` and `ContactComponent` provide seamless devotee booking for Havans, Kundali consultations, and Pujas:

- **Field Sanitization & Validation:** Enforces mandatory name, 10-digit Indian phone validation (`[0-9]{10}`), gotra/family members, and date selection.
- **Dynamic Service Binding:** Pre-selects the requested ritual through query parameter routing (e.g., `/booking?service=havan`).
- **User Feedback Modal:** Replaces standard page redirects with an in-place confirmation modal presenting a booking reference ID and temple contact coordinates.

6.2 Production Build & Netlify CI/CD Configuration

Angular 21 utilizes the new `@angular/build:application` builder powered by Vite and esbuild. Output files are generated in `dist/sakoon-temple/browser` .

🚀 **Netlify Configuration (netlify.toml)**

```
# Build settings
[build]
  command = "npm run build"
  publish = "dist/sakoon-temple/browser"

# Set required Node 22 environment
[build.environment]
  NODE_VERSION = "22"

# Prevent 404 on SPA page refresh
[[redirects]]
  from = "/*"
  to = "/index.html"
  status = 200
```

🛡️ **Solution for SPA 404 Routing Errors**

In a Single Page Application (SPA), routes like `/calendar` , `/contact` , or `/services` are virtual client routes handled by JavaScript.

When a user refreshes `/calendar` directly on Netlify, the server searches for a physical `calendar/index.html` file on disk. Without redirection rules, this triggers a `404 Not Found` error.

By defining `/* /index.html 200` in both `netlify.toml` and `public/_redirects` , Netlify forwards all subpaths to the main `index.html` with HTTP Status 200, allowing Angular Router to resolve the page seamlessly.

7. Project Directory Structure

```
Sakoon-Temple/ ├── .github/ # CI/CD Workflows ├── .nvmrc # Node Version Pinning (Node 22) ├── netlify.toml # Netlify Build, Node Runtime & SPA Rewrites ├── angular.json # Angular Builder Configuration & Asset Globs ├── package.json # Dependencies & Build Scripts ├── public/ | ├── _redirects # Fallback SPA 200 Rewrite for Netlify | └── favicon.ico └── src/ ├── index.html # Root HTML shell ├── styles.css # Global CSS tokens, forms, responsive utility classes └── app/ ├── app.routes.ts # SPA Route Map ├── components/ | ├── header/ # Sticky Glassmorphism Header with Shimmer Beam | └── footer/ # Vedic Footer with Quick Links & Darshan Hours ├── pages/ | ├── home/ # Hero Slider (2-sec), Services Showcase & Darshan Status | ├── calendar/ # Full Panchang Grid, Filter Bar & Detail Modal | ├── booking/ # Vedic Appointment Booking & Gotra Form | ├── contact/ # Devotee Query & Location Information | └── services-detail/ # Detailed Puja & Havan Scriptural Significance ├── models/ | └── temple.models.ts # TypeScript Interfaces for Panchang, Timings & Services └── services/ └── temple.service.ts # Lunisolar Engine, 1s Clock Pulse & Reactive Signals
```